COSAS A TENER EN CUENTA

Primera parte

- En las asociaciones, cuando no es lista, hay que meter el ID dentro del INSERT, con el mismo nombre que se llama en la tabla seguido de _id
- En las asociaciones, si es una lista, se creará una tabla nombre_de_la_tabla_nombre_de_la_variable
- Usar siempre camel_case
- Cuando el 1 está en el otro lado: NotNull

SERVICE

- Poner @Service arriba
- Poner @Autowired encima del repo (no hacer de primeras)
- El @Transactional(readOnly=true) para todo menos para el delete
- Si hay que lanzar una exception: @Transactional(rollbackofor = nombre_exception.class)

CONTROLLER

- Comenzar con @GetMapping(“url”)
- Si hay que meter algún id, dentro de la función (@Pathvariable Integer id)

```java
@RestController
public class ChessProblemAttemptController {

    UserService userService;
    ChessProblemAttemptService matchService;


    public ChessProblemAttemptController(ChessProblemAttemptService service, UserService userService){
        this.matchService=service;
        this.userService=userService;
    }

    @GetMapping("/api/v1/attempts")
    public List<ChessProblemAttempt> getAllAttempts(){
        return matchService.getAttempts();
    }

    @GetMapping("/api/v1/attempts/{id}")
    public ChessProblemAttempt getAttempt(@PathVariable Integer id) {
        Optional<ChessProblemAttempt> attempt = matchService.getMatchById(id);

        if (!attempt.isPresent()) {
            throw new ResourceNotFoundException("Attempt with id " + id + " not found");
        }

        return attempt.get();
    }

}
```

```java
@Service
public class ChessProblemAttemptService {

    ChessProblemAttemptRepository repo;

    @Autowired
    public ChessProblemAttemptService(ChessProblemAttemptRepository repo) {
        this.repo = repo;
    }

    @Transactional(readOnly = true)
    public Optional<ChessProblemAttempt> getMatchById(Integer id) {
        return repo.findById(id);
    }

    @Transactional(rollbackFor = UnfeasibleAttemptException.class)
    public ChessProblemAttempt save(ChessProblemAttempt m) throws UnfeasibleAttemptException {
        if( m.getStart() != null && m.getFinish() != null && m.getStart().isAfter(m.getFinish())){
            throw new UnfeasibleAttemptException(message: "Start time cannot be after finish time");
        }
        List<ChessProblemAttempt> activeAttempts = repo.findOngoingAttemptsByUserAndProblem(m.getPlayer(), m.getProblem());
        if(!activeAttempts.isEmpty() && !activeAttempts.get(index: 0).getId().equals(m.getId())){
            throw new UnfeasibleAttemptException(message: "There is already an active attempt for this user and problem");
        }
        return repo.save(m);
    }

    @Transactional(readOnly = true)
    public List<ChessProblemAttempt> getAttempts() {
        return repo.findAll();
    }

}
```

```java
public interface ChessProblemRepository extends CrudRepository<ChessProblem, Integer> {

    @Query("SELECT p FROM ChessProblem p WHERE (p.difficultyLevel >= :difficultyLevel) AND (size(p.board.pieces) < :numPieces)")
    List<ChessProblem> findByDifficultyAndNumPieces(Integer difficultyLevel, Integer numPieces);

    List<ChessProblem> findAll();

    Optional<ChessProblem> findById(Integer i);

    void delete(ChessProblem piece);

}
```

```java
@Entity
public class ChessProblemSolution extends BaseEntity {

    @NotNull
    @Size (min = 3, max = 50)
    @NotBlank
    String name;

    @NotNull
    LocalDateTime createdAt;

    @NotNull
    boolean minimal;

    @Size (max = 4000)
    String notes;

    @Min (1)
    @Max (15)
    Integer expectedMoveCount;

    @ManyToOne
    @NotNull
    ChessProblem problem;

    @OneToOne
    @NotNull
    ChessBoard solutionBoard;
}
```

```java
@Entity
public class ProblemSource extends BaseEntity {

    @NotNull
    @Size (min = 3, max = 50)
    @NotBlank
    String name;

    @NotNull
    @Size (min = 3, max = 100)
    String author;

    @Size (max = 1000)
    String description;

    @Email
    String contactEmail;

    @ManyToMany
    List<ChessProblem> problems;
}
```

```sql
-- ProblemSource for the problems
insert into problem_source (id, name, author, description, contact_email) values
  (801, 'Classic Puzzle Compendium', 'H. Steinitz', 'A curated set of elegant checkmate studies published in 1895.', 'steinitz@classicpuzzles.org'),
  (802, 'Modern Engine Analysis 2025', 'DeepEval AI', 'Computer-validated solutions using latest neural networks.', 'analysis@deepeval.ai');

-- ChessProblemSolution for the problems
insert into chess_problem_solution (id, name, created_at, minimal, notes, expected_move_count, problem_id, solution_board_id) values
  (901, 'Canonical Mate in 1 (line A)', '2025-01-10 09:11:00', true, 'Immediate mate with Qh5# from the starting position.', 1, 1, 910),
  (902, 'Canonical Mate in 2 (line B)', '2025-01-10 09:21:00', true, 'Two-move forcing sequence ending in mate with Qd5#.', 2, 3, 911);

-- problem-source-problems
insert into problem_source_problems (problem_source_id, problems_id) values
  (801, 1),
  (801, 3),
  (802, 2),
  (802, 4),
  (802, 5);
```